

CAMS

Computer Account Management System

USE CASE SPECIFICATIONS

219 use cases across 29 modules and 3 actors
Administrator 91 · Teacher 107 · Student 21

About this document

Each use case below is written in the ten fields prescribed for a use case specification: use case name, purpose, actors, input parameters, output parameters, pre-condition, post-condition, successful scenario, exception scenario and additional remarks.

Each use case is named after the function in the CAMS source that implements it. A specification titled `CREATE STUDENT` describes `AdminController.CreateStudent`; one titled `LOGIN ASYNC` describes `AuthenticationService.LoginAsync`, used where the behaviour has no endpoint of its own and is reached through an include relationship.

The fields that can be got wrong are read out of the source rather than written from memory. Input parameters come from the action signature and its `[Bind]` list, so a specification cannot name a field the form does not submit. The pre-condition comes from the `[Authorize]` attribute, including the `[TeacherSharedAction]` marker that admits a teacher to an administrator action. The antiforgery step appears only where the controller validates one, and a hub method is described as a SignalR relay rather than a form post, because that is what it is.

The companion diagram is `CAMS-Use-Case-Diagram.drawio`, in the same folder.

Admin

PROCESS LOG IN AccountController

A-001 LOGIN USER

Use Case Name	LOGIN USER
Purpose	Let a person sign in to the CAMS web portal with a username and password, and place them in the part of the system their role allows.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	username : string password : string
Output Parameters	An authentication cookie carrying the account role as a claim, and a redirect to the landing page for that role.
Pre-condition	None. This is the sign-in endpoint and is reachable without an account session; the CAMS server must be running and reachable.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the CAMS sign-in page. 2. The Admin enters a username and a password and submits the form. 3. The server validates the antiforgery token that accompanied the form. 4. The server checks the credentials against each account table in turn and finds the matching account. 5. The server confirms the account is active and not locked out. 6. The server issues an authentication cookie carrying the role as a claim. 7. The Admin is redirected to the landing page for that role.
Exception Scenario	• The username matches no account — the page reports that the sign-in failed, without saying which half was wrong. • The password does not match the stored hash — the failed-attempt counter is raised and the same message is shown. • The account is locked out — the sign-in is refused until the lockout expires, even with the right password. • The account is deactivated — the sign-in is refused and the person is told to contact an administrator.
Additional Remarks	• Implemented by <code>AccountController.Login</code> (POST). • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram.

	• Pulls in LOGIN ASYNC (<<include>>).
--	--

A-002 **LOGOUT USER**

Use Case Name	LOGOUT USER
Purpose	End the signed-in session and clear the authentication cookie, so the next visitor to the browser starts as an anonymous user.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The authentication cookie is cleared and the browser is returned to the sign-in page.
Pre-condition	The caller is signed in.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin chooses to sign out. 2. The server closes any lab session the account still has open. 3. The server clears the authentication cookie. 4. The browser is returned to the sign-in page as an anonymous visitor.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AccountController.Logout</code> (POST). • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Pulls in LOGOUT ASYNC (<<include>>).

A-003 **LOGIN ASYNC**

Use Case Name	LOGIN ASYNC
Purpose	Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside LOGIN USER
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A <code>LoginResult</code> naming the role, the account id and the display name, or a result saying the credentials were rejected.
Pre-condition	The sign-in use case has supplied a username and a password. LOGIN USER has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The sign-in use case passes the username, the password, the workstation name and the caller address. 2. The service looks the username up in the administrator, teacher and student tables. 3. The stored hash is verified against the supplied password. 4. The active flag and the lockout expiry are checked. 5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>LoginAsync</code>. • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Drawn as <<include>> from LOGIN USER.

A-004 **LOGOUT ASYNC**

Use Case Name	LOGOUT ASYNC
Purpose	Close any lab session the student still has open and record the sign-out.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside LOGOUT USER
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. LOGOUT USER has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs LogoutAsync and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by LogoutAsync. • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Drawn as <<include>> from LOGOUT USER.

VIEW ADMIN HOME AdminController

A-005 ADMIN INDEX

Use Case Name	ADMIN INDEX
Purpose	Show the administrator landing page, summarising the state of the laboratory.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Index</code> (GET). • Appears in the <i>VIEW ADMIN HOME</i> module of the use case diagram.

A-006 **VIEW SETTINGS**

Use Case Name	VIEW SETTINGS
Purpose	Show the settings page for the signed-in user.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Settings (GET)</code>. • Appears in the <i>VIEW ADMIN HOME</i> module of the use case diagram.

A-007 LAN CONFIG

Use Case Name	LAN CONFIG
Purpose	Show the network configuration the server detected, as a read-only diagnostic.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.LanConfig (GET)</code>. • Appears in the <i>VIEW ADMIN HOME</i> module of the use case diagram.

A-008 **CHANGE PASSWORD**

Use Case Name	CHANGE PASSWORD
Purpose	Let the signed-in user replace their own password after proving they know the current one.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	input : PasswordChangeInput
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.ChangePassword (POST)</code>. • Appears in the <i>VIEW ADMIN HOME</i> module of the use case diagram. • Pulls in CHANGE ADMIN PASSWORD ASYNC (<<include>>).

A-009 **CHANGE ADMIN PASSWORD ASYNC**

Use Case Name	CHANGE ADMIN PASSWORD ASYNC
Purpose	Verify the current password and replace the stored hash for an administrator account.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside CHANGE PASSWORD
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. CHANGE PASSWORD has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>ChangeAdminPasswordAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>ChangeAdminPasswordAsync</code>. • Appears in the <i>VIEW ADMIN HOME</i> module of the use case diagram. • Drawn as <<include>> from CHANGE PASSWORD.

MANAGE ADMIN ACCOUNT AdminController

A-010 CREATE ADMIN

Use Case Name	CREATE ADMIN
Purpose	Record a new administrator account in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	admin : Admin
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.CreateAdmin</code> (POST). • Appears in the <i>MANAGE ADMIN ACCOUNT</i> module of the use case diagram.

A-011 **UPDATE ADMIN**

Use Case Name	UPDATE ADMIN
Purpose	Amend the stored details of an existing administrator account.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	admin : Admin newPassword : string?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateAdmin (POST). • Appears in the <i>MANAGE ADMIN ACCOUNT</i> module of the use case diagram.

A-012 **DELETE ADMIN**

Use Case Name	DELETE ADMIN
Purpose	Remove a administrator account from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteAdmin (POST). • Appears in the <i>MANAGE ADMIN ACCOUNT</i> module of the use case diagram.

MANAGE TEACHER ACCOUNT AdminController

A-013 VIEW TEACHERS

Use Case Name	VIEW TEACHERS
Purpose	List the teachers the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Teachers (GET)</code>. • Appears in the <i>MANAGE TEACHER ACCOUNT</i> module of the use case diagram.

A-014 **CREATE TEACHER**

Use Case Name	CREATE TEACHER
Purpose	Record a new teacher account in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	teacher : Teacher
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.CreateTeacher</code> (POST). • Appears in the <i>MANAGE TEACHER ACCOUNT</i> module of the use case diagram.

A-015 **UPDATE TEACHER**

Use Case Name	UPDATE TEACHER
Purpose	Amend the stored details of an existing teacher account.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	teacher : Teacher newPassword : string?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateTeacher (POST). • Appears in the <i>MANAGE TEACHER ACCOUNT</i> module of the use case diagram.

A-016 **DELETE TEACHER**

Use Case Name	DELETE TEACHER
Purpose	Remove a teacher account from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteTeacher (POST). • Appears in the <i>MANAGE TEACHER ACCOUNT</i> module of the use case diagram.

A-017 **UNLOCK ACCOUNT**

Use Case Name	UNLOCK ACCOUNT
Purpose	Clear the lockout on an account that has been locked by repeated failed sign-in attempts.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	accountRole : AccountRole id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.UnlockAccount (POST). • Appears in the <i>MANAGE TEACHER ACCOUNT</i> module of the use case diagram.

A-018 SET ACCOUNT ACTIVE

Use Case Name	SET ACCOUNT ACTIVE
Purpose	Activate or deactivate an account without deleting it, so a person can be kept out of the system while their records survive.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	accountRole : AccountRole id : int isActive : bool
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.SetAccountActive (POST). • Appears in the <i>MANAGE TEACHER ACCOUNT</i> module of the use case diagram.

MANAGE STUDENT ACCOUNT AdminController

A-019 VIEW STUDENTS

Use Case Name	VIEW STUDENTS
Purpose	List the students the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Students (GET)</code>. • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

A-020 **CREATE STUDENT**

Use Case Name	CREATE STUDENT
Purpose	Record a new student account in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	student : Student
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.CreateStudent</code> (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

A-021 **UPDATE STUDENT**

Use Case Name	UPDATE STUDENT
Purpose	Amend the stored details of an existing student account.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	student : Student newPassword : string?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateStudent (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

A-022 **DELETE STUDENT**

Use Case Name	DELETE STUDENT
Purpose	Remove a student account from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteStudent (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

A-023 **BULK CREATE STUDENTS**

Use Case Name	BULK CREATE STUDENTS
Purpose	Create many student accounts in one operation from pasted or uploaded CSV rows.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	bulkFirstNames : List<string>? bulkLastNames : List<string>? bulkUserNames : List<string>? bulkPasswords : List<string>?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.BulkCreateStudents (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram. • Pulls in BULK PREVIEW CSV (<<include>>).

A-024 **BULK PREVIEW CSV**

Use Case Name	BULK PREVIEW CSV
Purpose	Parse the submitted CSV and show what would be created, so mistakes are caught before any account exists.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside BULK CREATE STUDENTS
Input Parameters	classId : int file : IFormFile?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists. BULK CREATE STUDENTS has reached the point where this is always performed.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.BulkPreviewCsv (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

- Drawn as <<include>> from **BULK CREATE STUDENTS**.

MANAGE COMPUTER PROFILE AdminController

A-025 VIEW COMPUTERS

Use Case Name	VIEW COMPUTERS
Purpose	List the computers the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by AdminController.Computers (GET). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

A-026 **CREATE COMPUTER**

Use Case Name	CREATE COMPUTER
Purpose	Record a new workstation profile in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	computer : Computer
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.CreateComputer</code> (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

A-027 **UPDATE COMPUTER**

Use Case Name	UPDATE COMPUTER
Purpose	Amend the stored details of an existing workstation profile.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	computer : Computer
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.UpdateComputer</code> (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

A-028 **DELETE COMPUTER**

Use Case Name	DELETE COMPUTER
Purpose	Remove a workstation profile from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteComputer (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

A-029 ASSIGN COMPUTER

Use Case Name	ASSIGN COMPUTER
Purpose	Bind a workstation to a student so the workstation is recognised when that student signs in at it.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	studentId : int computerId : int?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.AssignComputer (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

A-030 **COMPUTER HISTORY**

Use Case Name	COMPUTER HISTORY
Purpose	Show the recorded status changes for a workstation, and who made each one.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.ComputerHistory (GET). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

MANAGE CLASS AdminController

A-031 VIEW CLASSES

Use Case Name	VIEW CLASSES
Purpose	List the classes the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by AdminController.Classes (GET). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

A-032 **CREATE CLASS**

Use Case Name	CREATE CLASS
Purpose	Record a new class in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	cls : Class
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

A-033 **UPDATE CLASS**

Use Case Name	UPDATE CLASS
Purpose	Amend the stored details of an existing class.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	cls : Class
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

A-034 **DELETE CLASS**

Use Case Name	DELETE CLASS
Purpose	Remove a class from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	classId : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

A-035 ASSIGN TEACHER

Use Case Name	ASSIGN TEACHER
Purpose	Put a teacher in charge of a class.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	classId : int teacherId : int?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.AssignTeacher (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

A-036 ARCHIVE CLASS

Use Case Name	ARCHIVE CLASS
Purpose	Take a class out of active use while keeping its roster and records.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.ArchiveClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

MANAGE CLASS ROSTER AdminController

A-037 CLASS DETAILS

Use Case Name	CLASS DETAILS
Purpose	Show one class with its roster and the students enrolled in it.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>AdminController.ClassDetails</code> (GET). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

A-038 ENROLL STUDENT

Use Case Name	ENROLL STUDENT
Purpose	Add an existing student to a class roster.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	classId : int studentId : int moveStudent : bool (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.EnrollStudent (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

A-039 **ENROLL STUDENTS**

Use Case Name	ENROLL STUDENTS
Purpose	Add several existing students to a class roster in one operation.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	classId : int studentIds : List<int>? moveStudent : bool (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.EnrollStudents (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

A-040 ADD STUDENT TO CLASS

Use Case Name	ADD STUDENT TO CLASS
Purpose	Create a new student account and place it on a class roster in one step.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	<pre> classId : int firstName : string lastName : string username : string? password : string? </pre>
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>AdminController.AddStudentToClass</code> (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

A-041 ASSIGN STUDENT TO CLASS

Use Case Name	ASSIGN STUDENT TO CLASS
Purpose	Set the primary class a student belongs to.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	studentId : int classId : int? moveStudent : bool (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.AssignStudentToClass (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

A-042 REMOVE STUDENT

Use Case Name	REMOVE STUDENT
Purpose	Take a student off a class roster while leaving the student account intact.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	classId : int studentId : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.RemoveStudent (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

MANAGE RESTRICTION RULE AdminController

A-043 VIEW RESTRICTIONS

Use Case Name	VIEW RESTRICTIONS
Purpose	List the restrictions the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Restrictions (GET)</code>. • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

A-044 **CREATE RESTRICTION**

Use Case Name	CREATE RESTRICTION
Purpose	Record a new restriction rule in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	rule : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateRestriction (POST). • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

A-045 **UPDATE RESTRICTION**

Use Case Name	UPDATE RESTRICTION
Purpose	Amend the stored details of an existing restriction rule.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	input : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateRestriction (POST). • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

A-046 **DELETE RESTRICTION**

Use Case Name	DELETE RESTRICTION
Purpose	Remove a restriction rule from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteRestriction (POST). • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

MANAGE BLACKLIST AND WHITELIST AdminController

A-047 VIEW BLACKLISTS

Use Case Name	VIEW BLACKLISTS
Purpose	List the blacklists the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Blacklists</code> (GET). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

A-048 **VIEW WHITELISTS**

Use Case Name	VIEW WHITELISTS
Purpose	List the whitelists the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Whitelists (GET)</code>. • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

A-049 **CREATE BLACKLIST**

Use Case Name	CREATE BLACKLIST
Purpose	Record a new blacklist entry in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	RestrictionRuleId — bound from the submitted form RuleType — bound from the submitted form Target — bound from the submitted form Description — bound from the submitted form IsGlobal — bound from the submitted form IsActive — bound from the submitted form item : BlacklistItem
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateBlacklist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

A-050 **UPDATE BLACKLIST**

Use Case Name	UPDATE BLACKLIST
Purpose	Amend the stored details of an existing blacklist entry.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	input : BlacklistItem
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateBlacklist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

A-051 **DELETE BLACKLIST**

Use Case Name	DELETE BLACKLIST
Purpose	Remove a blacklist entry from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteBlacklist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

A-052 **CREATE WHITELIST**

Use Case Name	CREATE WHITELIST
Purpose	Record a new whitelist entry in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	rule : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateWhitelist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

A-053 **UPDATE WHITELIST**

Use Case Name	UPDATE WHITELIST
Purpose	Amend the stored details of an existing whitelist entry.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	RuleType — bound from the submitted form Target — bound from the submitted form Description — bound from the submitted form IsGlobal — bound from the submitted form IsActive — bound from the submitted form rule : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateWhitelist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

MANAGE CATEGORY AdminController

A-054 CREATE APPLICATION CATEGORY

Use Case Name	CREATE APPLICATION CATEGORY
Purpose	Record a new application category in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	category : ApplicationCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateApplicationCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

A-055 **UPDATE APPLICATION CATEGORY**

Use Case Name	UPDATE APPLICATION CATEGORY
Purpose	Amend the stored details of an existing application category.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	input : ApplicationCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateApplicationCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

A-056 **DELETE APPLICATION CATEGORY**

Use Case Name	DELETE APPLICATION CATEGORY
Purpose	Remove a application category from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteApplicationCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

A-057 **CREATE WEBSITE CATEGORY**

Use Case Name	CREATE WEBSITE CATEGORY
Purpose	Record a new website category in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	category : WebsiteCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateWebsiteCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

A-058 **UPDATE WEBSITE CATEGORY**

Use Case Name	UPDATE WEBSITE CATEGORY
Purpose	Amend the stored details of an existing website category.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	input : WebsiteCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateWebsiteCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

A-059 **DELETE WEBSITE CATEGORY**

Use Case Name	DELETE WEBSITE CATEGORY
Purpose	Remove a website category from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteWebsiteCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

MANAGE SESSION RULE AdminController

A-060 SESSION RULES

Use Case Name	SESSION RULES
Purpose	List the session rules the signed-in user is allowed to see.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.SessionRules (GET)</code>. • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

A-061 **CREATE SESSION RULE**

Use Case Name	CREATE SESSION RULE
Purpose	Record a new session rule in the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	rule : SessionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateSessionRule (POST). • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

A-062 **UPDATE SESSION RULE**

Use Case Name	UPDATE SESSION RULE
Purpose	Amend the stored details of an existing session rule.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	input : SessionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateSessionRule (POST). • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

A-063 **DELETE SESSION RULE**

Use Case Name	DELETE SESSION RULE
Purpose	Remove a session rule from the system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteSessionRule (POST). • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

MANAGE ROLE AND PERMISSION AdminController

A-064 VIEW ROLES

Use Case Name	VIEW ROLES
Purpose	Show the seeded roles and the permissions attached to them.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Roles</code> (GET). • Appears in the <i>MANAGE ROLE AND PERMISSION</i> module of the use case diagram.

A-065 CREATE ROLE

Use Case Name	CREATE ROLE
Purpose	Add a role to the seeded reference data.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	role : Role
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateRole (POST). • Appears in the <i>MANAGE ROLE AND PERMISSION</i> module of the use case diagram.

A-066 **DELETE ROLE**

Use Case Name	DELETE ROLE
Purpose	Remove a role from the seeded reference data.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	<code>id : int</code>
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by <code>AdminController.DeleteRole</code> (POST). • Appears in the <i>MANAGE ROLE AND PERMISSION</i> module of the use case diagram.

CONTROL LABORATORY SESSION AdminController

A-067 PAUSE ALL SESSIONS

Use Case Name	PAUSE ALL SESSIONS
Purpose	Pause every active laboratory session at once, freezing the timers across the room.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.PauseAllSessions</code> (POST). • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram.

A-068 RESUME ALL SESSIONS

Use Case Name	RESUME ALL SESSIONS
Purpose	Resume every paused laboratory session at once.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.ResumeAllSessions</code> (POST). • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram.

A-069 **END ALL SESSIONS**

Use Case Name	END ALL SESSIONS
Purpose	End every active laboratory session at once and tell the workstations.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.EndAllSessions</code> (POST). • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram. • Pulls in NOTIFY STATES ASYNC (<<include>>).

A-070 NOTIFY STATES ASYNC

Use Case Name	NOTIFY STATES ASYNC
Purpose	Push the new session state to every affected workstation so clients stop their timers rather than waiting to notice.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside END ALL SESSIONS
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. END ALL SESSIONS has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>NotifyStatesAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>NotifyStatesAsync</code>. • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram. • Drawn as <<include>> from END ALL SESSIONS.

VIEW REPORTS AND LOGS AdminController

A-071 VIEW REPORTS

Use Case Name	VIEW REPORTS
Purpose	Show the laboratory reports an administrator uses for oversight.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	from : DateTime? to : DateTime? classId : int? (optional) station : string? (optional) page : int (optional) pageSize : int (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.Reports (GET). • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Pulls in EXPORT REPORTS CSV (<<extend>>), EXPORT ATTENDANCE CSV (<<extend>>), EXPORT USAGE CSV (<<extend>>), EXPORT REMOTE COMMANDS CSV (<<extend>>).

A-072 **AUDIT LOGS**

Use Case Name	AUDIT LOGS
Purpose	Show the audit trail of administrative actions.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.AuditLogs (GET)</code>. • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Pulls in EXPORT AUDIT CSV (<<extend>>).

A-073 **SYSTEM LOGS**

Use Case Name	SYSTEM LOGS
Purpose	Show the technical log the server writes.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.SystemLogs (GET)</code>. • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Pulls in EXPORT SYSTEM LOGS CSV (<<extend>>).

A-074 **EXPORT REPORTS CSV**

Use Case Name	EXPORT REPORTS CSV
Purpose	Produce the reports view as a CSV file the user can download.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside VIEW REPORTS
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) classId : int? (optional) station : string? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. The record named by the identifier exists. VIEW REPORTS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>AdminController.ExportReportsCsv</code> (GET). • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Drawn as <<extend>> to VIEW REPORTS.

A-075 **EXPORT ATTENDANCE CSV**

Use Case Name	EXPORT ATTENDANCE CSV
Purpose	Produce the attendance view as a CSV file the user can download.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside VIEW REPORTS
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) classId : int? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. The record named by the identifier exists. VIEW REPORTS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>AdminController.ExportAttendanceCsv (GET)</code>. • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Drawn as <<extend>> to VIEW REPORTS.

A-076 **EXPORT USAGE CSV**

Use Case Name	EXPORT USAGE CSV
Purpose	Produce the usage view as a CSV file the user can download.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside VIEW REPORTS
Input Parameters	from : DateTime? to : DateTime?
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. VIEW REPORTS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.ExportUsageCsv (GET)</code>. • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Drawn as <<extend>> to VIEW REPORTS.

A-077 **EXPORT REMOTE COMMANDS CSV**

Use Case Name	EXPORT REMOTE COMMANDS CSV
Purpose	Produce the remote commands view as a CSV file the user can download.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside VIEW REPORTS
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) teacherId : int? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. The record named by the identifier exists. VIEW REPORTS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.ExportRemoteCommandsCsv (GET). • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Drawn as <<extend>> to VIEW REPORTS.

A-078 **EXPORT AUDIT CSV**

Use Case Name	EXPORT AUDIT CSV
Purpose	Produce the audit view as a CSV file the user can download.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside AUDIT LOGS
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. AUDIT LOGS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.ExportAuditCsv (GET)</code>. • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Drawn as <<extend>> to AUDIT LOGS.

A-079 **EXPORT SYSTEM LOGS CSV**

Use Case Name	EXPORT SYSTEM LOGS CSV
Purpose	Produce the system logs view as a CSV file the user can download.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside SYSTEM LOGS
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. SYSTEM LOGS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.ExportSystemLogsCsv (GET)</code>. • Appears in the <i>VIEW REPORTS AND LOGS</i> module of the use case diagram. • Drawn as <<extend>> to SYSTEM LOGS.

MANAGE DATABASE AdminDatabaseController

A-080 DATABASE INDEX

Use Case Name	DATABASE INDEX
Purpose	Show database health: file size, integrity check, applied and pending migrations, and the backups on disk.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminDatabaseController.Index (GET)</code>. • Appears in the <i>MANAGE DATABASE</i> module of the use case diagram.

A-081 CREATE BACKUP

Use Case Name	CREATE BACKUP
Purpose	Take a backup copy of the SQLite database file and store it on the server.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	label : string?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminDatabaseController.CreateBackup</code> (POST). • Appears in the <i>MANAGE DATABASE</i> module of the use case diagram.

A-082 **VALIDATE BACKUP**

Use Case Name	VALIDATE BACKUP
Purpose	Check that a backup file is a readable, intact database before anybody relies on it.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	backupFileName : string
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminDatabaseController.ValidateBackup (POST). • Appears in the <i>MANAGE DATABASE</i> module of the use case diagram.

A-083 **STAGE RESTORE**

Use Case Name	STAGE RESTORE
Purpose	Stage a backup so the server restores it on the next restart, rather than swapping the file underneath a running system.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	backupFileName : string confirmation : string?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminDatabaseController.StageRestore</code> (POST). • Appears in the <i>MANAGE DATABASE</i> module of the use case diagram. • Pulls in STAGE RESTORE ASYNC (<<include>>).

A-084 **STAGE RESTORE ASYNC**

Use Case Name	STAGE RESTORE ASYNC
Purpose	Take a safety backup of the current database and mark the chosen backup for restore at next start.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside STAGE RESTORE
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. STAGE RESTORE has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs StageRestoreAsync and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by StageRestoreAsync. • Appears in the <i>MANAGE DATABASE</i> module of the use case diagram. • Drawn as <<include>> from STAGE RESTORE.

MANAGE DEPLOYMENT AdminDeploymentController

A-085 DEPLOYMENT INDEX

Use Case Name	DEPLOYMENT INDEX
Purpose	Show the deployment hub: release version, installer and certificate state, and the endpoint clients should use.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminDeploymentController.Index (GET)</code>. • Appears in the <i>MANAGE DEPLOYMENT</i> module of the use case diagram. • Pulls in GET STATUS ASYNC (<<include>>).

A-086 **DOWNLOAD INSTALLER**

Use Case Name	DOWNLOAD INSTALLER
Purpose	Download the client installer for deployment to a workstation.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminDeploymentController.Installer (GET)</code>. • Appears in the <i>MANAGE DEPLOYMENT</i> module of the use case diagram.

A-087 **DOWNLOAD MANIFEST**

Use Case Name	DOWNLOAD MANIFEST
Purpose	Download the release manifest listing the files and their hashes.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminDeploymentController.Manifest (GET)</code>. • Appears in the <i>MANAGE DEPLOYMENT</i> module of the use case diagram.

A-088 **ROOT CERTIFICATE**

Use Case Name	ROOT CERTIFICATE
Purpose	Download the root certificate a workstation must trust to reach the server over HTTPS.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminDeploymentController.RootCertificate (GET)</code>. • Appears in the <i>MANAGE DEPLOYMENT</i> module of the use case diagram.

A-089 CREATE BUNDLE

Use Case Name	CREATE BUNDLE
Purpose	Build and download an offline client bundle for a workstation with no access to the server yet.
Primary Actor	Admin
Secondary Actor	None
Input Parameters	endpoint : string
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Admin opens the page and CAMS confirms the role on the authentication cookie. 2. The Admin fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminDeploymentController.Bundle (POST). • Appears in the <i>MANAGE DEPLOYMENT</i> module of the use case diagram. • Pulls in CREATE BUNDLE ASYNC (<<include>>).

A-090 **GET STATUS ASYNC**

Use Case Name	GET STATUS ASYNC
Purpose	Gather the deployment state: release version, file hashes, certificate and endpoint.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside DEPLOYMENT INDEX
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. DEPLOYMENT INDEX has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>GetStatusAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>GetStatusAsync</code>. • Appears in the <i>MANAGE DEPLOYMENT</i> module of the use case diagram. • Drawn as <<include>> from DEPLOYMENT INDEX.

A-091 **CREATE BUNDLE ASYNC**

Use Case Name	CREATE BUNDLE ASYNC
Purpose	Assemble an offline installer bundle carrying the installer, the root certificate and the server endpoint.
Primary Actor	Admin
Secondary Actor	None; this behaviour runs inside CREATE BUNDLE
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. CREATE BUNDLE has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>CreateBundleAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>CreateBundleAsync</code>. • Appears in the <i>MANAGE DEPLOYMENT</i> module of the use case diagram. • Drawn as <<include>> from CREATE BUNDLE.

Teacher

PROCESS LOG IN AccountController

T-092 LOGIN USER

Use Case Name	LOGIN USER
Purpose	Let a person sign in to the CAMS web portal with a username and password, and place them in the part of the system their role allows.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	username : string password : string
Output Parameters	An authentication cookie carrying the account role as a claim, and a redirect to the landing page for that role.
Pre-condition	None. This is the sign-in endpoint and is reachable without an account session; the CAMS server must be running and reachable.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the CAMS sign-in page. 2. The Teacher enters a username and a password and submits the form. 3. The server validates the antiforgery token that accompanied the form. 4. The server checks the credentials against each account table in turn and finds the matching account. 5. The server confirms the account is active and not locked out. 6. The server issues an authentication cookie carrying the role as a claim. 7. The Teacher is redirected to the landing page for that role.
Exception Scenario	• The username matches no account — the page reports that the sign-in failed, without saying which half was wrong. • The password does not match the stored hash — the failed-attempt counter is raised and the same message is shown. • The account is locked out — the sign-in is refused until the lockout expires, even with the right password. • The account is deactivated — the sign-in is refused and the person is told to contact an administrator.
Additional Remarks	• Implemented by <code>AccountController.Login</code> (POST). • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram.

	• Pulls in LOGIN ASYNC (<<include>>).
--	--

T-093 **LOGOUT USER**

Use Case Name	LOGOUT USER
Purpose	End the signed-in session and clear the authentication cookie, so the next visitor to the browser starts as an anonymous user.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The authentication cookie is cleared and the browser is returned to the sign-in page.
Pre-condition	The caller is signed in.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher chooses to sign out. 2. The server closes any lab session the account still has open. 3. The server clears the authentication cookie. 4. The browser is returned to the sign-in page as an anonymous visitor.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AccountController.Logout</code> (POST). • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Pulls in LOGOUT ASYNC (<<include>>).

T-094 **LOGIN ASYNC**

Use Case Name	LOGIN ASYNC
Purpose	Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside LOGIN USER
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A <code>LoginResult</code> naming the role, the account id and the display name, or a result saying the credentials were rejected.
Pre-condition	The sign-in use case has supplied a username and a password. LOGIN USER has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The sign-in use case passes the username, the password, the workstation name and the caller address. 2. The service looks the username up in the administrator, teacher and student tables. 3. The stored hash is verified against the supplied password. 4. The active flag and the lockout expiry are checked. 5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>LoginAsync</code>. • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Drawn as <<include>> from LOGIN USER.

T-095 **LOGOUT ASYNC**

Use Case Name	LOGOUT ASYNC
Purpose	Close any lab session the student still has open and record the sign-out.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside LOGOUT USER
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. LOGOUT USER has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs LogoutAsync and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by LogoutAsync. • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Drawn as <<include>> from LOGOUT USER.

MANAGE OWN ACCOUNT TeacherController

T-096 VIEW SETTINGS

Use Case Name	VIEW SETTINGS
Purpose	Show the settings page for the signed-in user.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Settings (GET)</code>. • Appears in the <i>MANAGE OWN ACCOUNT</i> module of the use case diagram.

T-097 **CHANGE PASSWORD**

Use Case Name	CHANGE PASSWORD
Purpose	Let the signed-in user replace their own password after proving they know the current one.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	input : PasswordChangeInput
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>TeacherController.ChangePassword</code> (POST). • Appears in the <i>MANAGE OWN ACCOUNT</i> module of the use case diagram. • Pulls in CHANGE TEACHER PASSWORD ASYNC (<<include>>).

T-098 **CHANGE TEACHER PASSWORD ASYNC**

Use Case Name	CHANGE TEACHER PASSWORD ASYNC
Purpose	Verify the current password and replace the stored hash for a teacher account.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside CHANGE PASSWORD
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. CHANGE PASSWORD has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>ChangeTeacherPasswordAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>ChangeTeacherPasswordAsync</code>. • Appears in the <i>MANAGE OWN ACCOUNT</i> module of the use case diagram. • Drawn as <<include>> from CHANGE PASSWORD.

MANAGE PEER TEACHER ACCOUNT AdminController

T-099 VIEW TEACHERS

Use Case Name	VIEW TEACHERS
Purpose	List the teachers the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Teachers (GET)</code>. • Appears in the <i>MANAGE PEER TEACHER ACCOUNT</i> module of the use case diagram.

T-100 CREATE TEACHER

Use Case Name	CREATE TEACHER
Purpose	Record a new teacher account in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	teacher : Teacher
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.CreateTeacher</code> (POST). • Appears in the <i>MANAGE PEER TEACHER ACCOUNT</i> module of the use case diagram.

T-101 UPDATE TEACHER

Use Case Name	UPDATE TEACHER
Purpose	Amend the stored details of an existing teacher account.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	teacher : Teacher newPassword : string?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateTeacher (POST). • Appears in the <i>MANAGE PEER TEACHER ACCOUNT</i> module of the use case diagram.

T-102 **DELETE TEACHER**

Use Case Name	DELETE TEACHER
Purpose	Remove a teacher account from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteTeacher (POST). • Appears in the <i>MANAGE PEER TEACHER ACCOUNT</i> module of the use case diagram.

T-103 UNLOCK ACCOUNT

Use Case Name	UNLOCK ACCOUNT
Purpose	Clear the lockout on an account that has been locked by repeated failed sign-in attempts.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	accountRole : AccountRole id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.UnlockAccount (POST). • Appears in the <i>MANAGE PEER TEACHER ACCOUNT</i> module of the use case diagram.

T-104 SET ACCOUNT ACTIVE

Use Case Name	SET ACCOUNT ACTIVE
Purpose	Activate or deactivate an account without deleting it, so a person can be kept out of the system while their records survive.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	accountRole : AccountRole id : int isActive : bool
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.SetAccountActive (POST). • Appears in the <i>MANAGE PEER TEACHER ACCOUNT</i> module of the use case diagram.

MANAGE STUDENT ACCOUNT TeacherController

T-105 VIEW STUDENTS

Use Case Name	VIEW STUDENTS
Purpose	List the students the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	search : string? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Students (GET)</code>. • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

T-106 CREATE STUDENT

Use Case Name	CREATE STUDENT
Purpose	Record a new student account in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	student : Student classId : int? (optional) search : string? (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.CreateStudent</code> (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

T-107 UPDATE STUDENT

Use Case Name	UPDATE STUDENT
Purpose	Amend the stored details of an existing student account.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	student : Student newPassword : string? search : string? (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>TeacherController.UpdateStudent</code> (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

T-108 DELETE STUDENT

Use Case Name	DELETE STUDENT
Purpose	Remove a student account from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	studentId : int search : string? (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by <code>TeacherController.DeleteStudent</code> (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

T-109 **BULK ADD STUDENTS**

Use Case Name	BULK ADD STUDENTS
Purpose	Create many student accounts at once and enrol them into a class in the same operation.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	classId : int bulkFirstNames : List<string>? bulkLastNames : List<string>? bulkUserNames : List<string>? bulkPasswords : List<string>?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.

Additional Remarks	• Implemented by <code>TeacherController.BulkAddStudents</code> (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram. • Pulls in BULK PREVIEW CSV (<<include>>).
---------------------------	--

T-110 **BULK PREVIEW CSV**

Use Case Name	BULK PREVIEW CSV
Purpose	Parse the submitted CSV and show what would be created, so mistakes are caught before any account exists.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside BULK ADD STUDENTS
Input Parameters	classId : int file : IFormFile?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists. BULK ADD STUDENTS has reached the point where this is always performed.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.BulkPreviewCsv</code> (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram. • Drawn as <<include>> from BULK ADD STUDENTS.

MANAGE COMPUTER PROFILE AdminController

T-111 VIEW COMPUTERS

Use Case Name	VIEW COMPUTERS
Purpose	List the computers the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by AdminController.Computers (GET). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

T-112 **CREATE COMPUTER**

Use Case Name	CREATE COMPUTER
Purpose	Record a new workstation profile in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	computer : Computer
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AdminController.CreateComputer</code> (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

T-113 UPDATE COMPUTER

Use Case Name	UPDATE COMPUTER
Purpose	Amend the stored details of an existing workstation profile.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	computer : Computer
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateComputer (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

T-114 **DELETE COMPUTER**

Use Case Name	DELETE COMPUTER
Purpose	Remove a workstation profile from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteComputer (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

T-115 ASSIGN COMPUTER

Use Case Name	ASSIGN COMPUTER
Purpose	Bind a workstation to a student so the workstation is recognised when that student signs in at it.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	studentId : int computerId : int?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.AssignComputer (POST). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

T-116 **COMPUTER HISTORY**

Use Case Name	COMPUTER HISTORY
Purpose	Show the recorded status changes for a workstation, and who made each one.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by AdminController.ComputerHistory (GET). • Appears in the <i>MANAGE COMPUTER PROFILE</i> module of the use case diagram.

MANAGE CLASS TeacherController

T-117 VIEW CLASSES

Use Case Name	VIEW CLASSES
Purpose	List the classes the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Classes</code> (GET). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

T-118 CREATE CLASS

Use Case Name	CREATE CLASS
Purpose	Record a new class in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	cls : Class
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by TeacherController.CreateClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

T-119 UPDATE CLASS

Use Case Name	UPDATE CLASS
Purpose	Amend the stored details of an existing class.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	cls : Class
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by TeacherController.UpdateClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

T-120 **DELETE CLASS**

Use Case Name	DELETE CLASS
Purpose	Remove a class from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	classId : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by TeacherController.DeleteClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

T-121 ASSIGN TEACHER

Use Case Name	ASSIGN TEACHER
Purpose	Put a teacher in charge of a class.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	classId : int teacherId : int?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.AssignTeacher</code> (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

T-122 ARCHIVE CLASS

Use Case Name	ARCHIVE CLASS
Purpose	Take a class out of active use while keeping its roster and records.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by TeacherController.ArchiveClass (POST). • Appears in the <i>MANAGE CLASS</i> module of the use case diagram.

MANAGE CLASS ROSTER TeacherController

T-123 CLASS DETAILS

Use Case Name	CLASS DETAILS
Purpose	Show one class with its roster and the students enrolled in it.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.ClassDetails</code> (GET). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

T-124 **ENROLL STUDENT**

Use Case Name	ENROLL STUDENT
Purpose	Add an existing student to a class roster.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	classId : int studentId : int moveStudent : bool (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.EnrollStudent</code> (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

T-125 **ENROLL STUDENTS**

Use Case Name	ENROLL STUDENTS
Purpose	Add several existing students to a class roster in one operation.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	classId : int studentIds : List<int>? moveStudent : bool (optional)
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.EnrollStudents</code> (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

T-126 ADD STUDENT TO CLASS

Use Case Name	ADD STUDENT TO CLASS
Purpose	Create a new student account and place it on a class roster in one step.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	classId : int firstName : string lastName : string username : string? password : string?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by TeacherController.AddStudentToClass (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

T-127 REMOVE STUDENT

Use Case Name	REMOVE STUDENT
Purpose	Take a student off a class roster while leaving the student account intact.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	classId : int studentId : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by TeacherController.RemoveStudent (POST). • Appears in the <i>MANAGE CLASS ROSTER</i> module of the use case diagram.

MANAGE RESTRICTION RULE `TeacherController`

T-128 VIEW RESTRICTIONS

Use Case Name	VIEW RESTRICTIONS
Purpose	List the restrictions the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Restrictions (GET)</code>. • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

T-129 **CREATE RESTRICTION**

Use Case Name	CREATE RESTRICTION
Purpose	Record a new restriction rule in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	rule : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by TeacherController.CreateRestriction (POST). • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

T-130 UPDATE RESTRICTION

Use Case Name	UPDATE RESTRICTION
Purpose	Amend the stored details of an existing restriction rule.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	input : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by TeacherController.UpdateRestriction (POST). • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

T-131 DELETE RESTRICTION

Use Case Name	DELETE RESTRICTION
Purpose	Remove a restriction rule from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by TeacherController.DeleteRestriction (POST). • Appears in the <i>MANAGE RESTRICTION RULE</i> module of the use case diagram.

MANAGE BLACKLIST AND WHITELIST AdminController

T-132 VIEW BLACKLISTS

Use Case Name	VIEW BLACKLISTS
Purpose	List the blacklists the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Blacklists</code> (GET). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

T-133 **VIEW WHITELISTS**

Use Case Name	VIEW WHITELISTS
Purpose	List the whitelists the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Whitelists (GET)</code>. • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

T-134 **CREATE BLACKLIST**

Use Case Name	CREATE BLACKLIST
Purpose	Record a new blacklist entry in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	RestrictionRuleId — bound from the submitted form RuleType — bound from the submitted form Target — bound from the submitted form Description — bound from the submitted form IsGlobal — bound from the submitted form IsActive — bound from the submitted form item : BlacklistItem
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateBlacklist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

T-135 **UPDATE BLACKLIST**

Use Case Name	UPDATE BLACKLIST
Purpose	Amend the stored details of an existing blacklist entry.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	input : BlacklistItem
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateBlacklist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

T-136 **DELETE BLACKLIST**

Use Case Name	DELETE BLACKLIST
Purpose	Remove a blacklist entry from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteBlacklist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

T-137 CREATE WHITELIST

Use Case Name	CREATE WHITELIST
Purpose	Record a new whitelist entry in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	rule : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateWhitelist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

T-138 **UPDATE WHITELIST**

Use Case Name	UPDATE WHITELIST
Purpose	Amend the stored details of an existing whitelist entry.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	RuleType — bound from the submitted form Target — bound from the submitted form Description — bound from the submitted form IsGlobal — bound from the submitted form IsActive — bound from the submitted form rule : RestrictionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateWhitelist (POST). • Appears in the <i>MANAGE BLACKLIST AND WHITELIST</i> module of the use case diagram.

MANAGE CATEGORY AdminController

T-139 CREATE APPLICATION CATEGORY

Use Case Name	CREATE APPLICATION CATEGORY
Purpose	Record a new application category in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	category : ApplicationCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateApplicationCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

T-140 **UPDATE APPLICATION CATEGORY**

Use Case Name	UPDATE APPLICATION CATEGORY
Purpose	Amend the stored details of an existing application category.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	input : ApplicationCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateApplicationCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

T-141 **DELETE APPLICATION CATEGORY**

Use Case Name	DELETE APPLICATION CATEGORY
Purpose	Remove a application category from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteApplicationCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

T-142 **CREATE WEBSITE CATEGORY**

Use Case Name	CREATE WEBSITE CATEGORY
Purpose	Record a new website category in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	category : WebsiteCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateWebsiteCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

T-143 **UPDATE WEBSITE CATEGORY**

Use Case Name	UPDATE WEBSITE CATEGORY
Purpose	Amend the stored details of an existing website category.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	input : WebsiteCategory
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateWebsiteCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

T-144 **DELETE WEBSITE CATEGORY**

Use Case Name	DELETE WEBSITE CATEGORY
Purpose	Remove a website category from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteWebsiteCategory (POST). • Appears in the <i>MANAGE CATEGORY</i> module of the use case diagram.

MANAGE SESSION RULE AdminController

T-145 SESSION RULES

Use Case Name	SESSION RULES
Purpose	List the session rules the signed-in user is allowed to see.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.SessionRules (GET)</code>. • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

T-146 CREATE SESSION RULE

Use Case Name	CREATE SESSION RULE
Purpose	Record a new session rule in the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	rule : SessionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.CreateSessionRule (POST). • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

T-147 **UPDATE SESSION RULE**

Use Case Name	UPDATE SESSION RULE
Purpose	Amend the stored details of an existing session rule.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	input : SessionRule
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by AdminController.UpdateSessionRule (POST). • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

T-148 **DELETE SESSION RULE**

Use Case Name	DELETE SESSION RULE
Purpose	Remove a session rule from the system.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change. • The record is still referenced — the deletion is refused rather than leaving dangling references.
Additional Remarks	• Implemented by AdminController.DeleteSessionRule (POST). • Appears in the <i>MANAGE SESSION RULE</i> module of the use case diagram.

CONTROL LABORATORY SESSION TeacherController

T-149 GLOBAL START SESSION

Use Case Name	GLOBAL START SESSION
Purpose	Start a laboratory-wide session so every connected workstation begins at the same moment.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>TeacherController.GlobalStartSession</code> (POST). • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram.

T-150 GLOBAL PAUSE SESSION

Use Case Name	GLOBAL PAUSE SESSION
Purpose	Pause the laboratory-wide session for the whole room.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>TeacherController.GlobalPauseSession</code> (POST). • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram.

T-151 GLOBAL END SESSION

Use Case Name	GLOBAL END SESSION
Purpose	End the laboratory-wide session for the whole room.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>TeacherController.GlobalEndSession</code> (POST). • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram. • Pulls in NOTIFY STATES ASYNC (<<include>>).

T-152 GLOBAL SESSION STATE

Use Case Name	GLOBAL SESSION STATE
Purpose	Report the current laboratory-wide session state so the page can show the right timer and controls.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.GlobalSessionState (GET)</code>. • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram.

T-153 NOTIFY STATES ASYNC

Use Case Name	NOTIFY STATES ASYNC
Purpose	Push the new session state to every affected workstation so clients stop their timers rather than waiting to notice.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside GLOBAL END SESSION
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. GLOBAL END SESSION has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>NotifyStatesAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>NotifyStatesAsync</code>. • Appears in the <i>CONTROL LABORATORY SESSION</i> module of the use case diagram. • Drawn as <<include>> from GLOBAL END SESSION.

CONTROL STUDENT SESSION TeacherController

T-154 VIEW SESSIONS

Use Case Name	VIEW SESSIONS
Purpose	List the lab sessions the teacher may act on.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Sessions</code> (GET). • Appears in the <i>CONTROL STUDENT SESSION</i> module of the use case diagram.

T-155 **START SESSION**

Use Case Name	START SESSION
Purpose	Open a lab session for one student at one workstation under a chosen session rule.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	studentId : int computerId : int? sessionRuleId : int?
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.StartSession</code> (POST). • Appears in the <i>CONTROL STUDENT SESSION</i> module of the use case diagram. • Pulls in ENSURE STUDENT SESSION ASYNC (<<include>>).

T-156 **TOGGLE PAUSE**

Use Case Name	TOGGLE PAUSE
Purpose	Pause a running session, or resume a paused one, accumulating the paused time so it is not charged against the limit.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.TogglePause</code> (POST). • Appears in the <i>CONTROL STUDENT SESSION</i> module of the use case diagram.

T-157 **END SESSION**

Use Case Name	END SESSION
Purpose	Close one student lab session and record its end time.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.EndSession (POST)</code>. • Appears in the <i>CONTROL STUDENT SESSION</i> module of the use case diagram.

T-158 ENSURE STUDENT SESSION ASYNC

Use Case Name	ENSURE STUDENT SESSION ASYNC
Purpose	Find the student a session, creating one if none is open, and apply the governing session rule to it.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside START SESSION
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. START SESSION has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>EnsureStudentSessionAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>EnsureStudentSessionAsync</code>. • Appears in the <i>CONTROL STUDENT SESSION</i> module of the use case diagram. • Drawn as <<include>> from START SESSION. • Pulls in END EXPIRED SESSIONS ASYNC (<<extend>>).

T-159 **END EXPIRED SESSIONS ASYNC**

Use Case Name	END EXPIRED SESSIONS ASYNC
Purpose	Close sessions that have run past the limit their session rule sets, without anyone having to intervene.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside ENSURE STUDENT SESSION ASYNC
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. ENSURE STUDENT SESSION ASYNC has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs EndExpiredSessionsAsync and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by EndExpiredSessionsAsync. • Appears in the <i>CONTROL STUDENT SESSION</i> module of the use case diagram. • Drawn as <<extend>> to ENSURE STUDENT SESSION ASYNC.

MONITOR STUDENT SCREEN TeacherController + Hub

T-160 OPEN MONITORING

Use Case Name	OPEN MONITORING
Purpose	Open the live monitoring wall showing every connected student workstation.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Monitoring (GET)</code>. • Appears in the <i>MONITOR STUDENT SCREEN</i> module of the use case diagram. • Pulls in SEND SCREEN FRAME (<<include>>).

T-161 **MONITORING INDEX**

Use Case Name	MONITORING INDEX
Purpose	Serve the monitoring surface the live screen grid is built on.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as an administrator or a teacher. A teacher reaches this action only because it carries [TeacherSharedAction]; the teacher account must also be active.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>AdminController.Index (GET)</code>. • Appears in the <i>MONITOR STUDENT SCREEN</i> module of the use case diagram.

T-162 **LIVE STATE**

Use Case Name	LIVE STATE
Purpose	Report the current state of every connected workstation so the monitoring page can refresh without reloading.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.LiveState (GET)</code>. • Appears in the <i>MONITOR STUDENT SCREEN</i> module of the use case diagram.

T-163 **SEND SCREEN FRAME**

Use Case Name	SEND SCREEN FRAME
Purpose	Send one captured frame of the workstation screen to the watching teacher.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	frame : ScreenFrameMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. OPEN MONITORING has reached the point where this is always performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes SendScreenFrame on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.SendScreenFrame (GET). • Appears in the <i>MONITOR STUDENT SCREEN</i> module of the use case diagram. • Drawn as <<include>> from OPEN MONITORING.

CONTROL STUDENT WORKSTATION RemoteMonitoringHub

T-164 START REMOTE CONTROL

Use Case Name	START REMOTE CONTROL
Purpose	Take keyboard and mouse control of a student workstation, with the client showing that remote control is active.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string
Output Parameters	A RemoteCommandResult stating whether the workstation accepted the command, returned to the caller over SignalR.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes StartRemoteControl on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.StartRemoteControl (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram. • Pulls in SEND REMOTE INPUT (<<include>>).

T-165 **STOP REMOTE CONTROL**

Use Case Name	STOP REMOTE CONTROL
Purpose	Hand control of the workstation back to the student.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string
Output Parameters	A RemoteCommandResult stating whether the workstation accepted the command, returned to the caller over SignalR.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes StopRemoteControl on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.StopRemoteControl (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-166 **LOCK STUDENT**

Use Case Name	LOCK STUDENT
Purpose	Lock a student workstation so the student cannot use it until it is unlocked.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes LockStudent on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.LockStudent (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-167 UNLOCK STUDENT

Use Case Name	UNLOCK STUDENT
Purpose	Release the CAMS lock on a student workstation.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes UnlockStudent on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.UnlockStudent (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-168 **FORCE LOGOUT**

Use Case Name	FORCE LOGOUT
Purpose	Sign a student out of the workstation from the teacher console.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes ForceLogout on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ForceLogout (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-169 SHUTDOWN STUDENT

Use Case Name	SHUTDOWN STUDENT
Purpose	Shut a student workstation down remotely.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes ShutdownStudent on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ShutdownStudent (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-170 RESTART STUDENT

Use Case Name	RESTART STUDENT
Purpose	Restart a student workstation remotely.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes RestartStudent on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.RestartStudent (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-171 **BULK LOCK STUDENTS**

Use Case Name	BULK LOCK STUDENTS
Purpose	Lock several student workstations in one action.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionIds : List<string>
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes BulkLockStudents on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.BulkLockStudents (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-172 **BULK FORCE LOGOUT STUDENTS**

Use Case Name	BULK FORCE LOGOUT STUDENTS
Purpose	Sign several students out of their workstations in one action.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionIds : List<string>
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes BulkForceLogoutStudents on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.BulkForceLogoutStudents (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram.

T-173 SEND REMOTE INPUT

Use Case Name	SEND REMOTE INPUT
Purpose	Deliver one keyboard or mouse event to the workstation under remote control.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string input : RemoteInputMessage
Output Parameters	A RemoteCommandResult stating whether the workstation accepted the command, returned to the caller over SignalR.
Pre-condition	The caller is signed in. The record named by the identifier exists. START REMOTE CONTROL has reached the point where this is always performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes SendRemoteInput on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state. 6. The command is written to the remote command log.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.SendRemoteInput (GET). • Appears in the <i>CONTROL STUDENT WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from START REMOTE CONTROL.

SEND MESSAGE TO STUDENT RemoteMonitoringHub

T-174 SEND NOTIFICATION

Use Case Name	SEND NOTIFICATION
Purpose	Send a message to connected students that appears on their workstation.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	notification : NotificationMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes SendNotification on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.SendNotification (GET). • Appears in the <i>SEND MESSAGE TO STUDENT</i> module of the use case diagram.

T-175 SEND WARNING POPUP

Use Case Name	SEND WARNING POPUP
Purpose	Send a warning dialog to one student, shown on top of whatever they are doing.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	targetConnectionId : string warning : NotificationMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. The record named by the identifier exists.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes SendWarningPopup on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.SendWarningPopup (GET). • Appears in the <i>SEND MESSAGE TO STUDENT</i> module of the use case diagram.

T-176 **BROADCAST SCREEN**

Use Case Name	BROADCAST SCREEN
Purpose	Put the teacher screen on every connected student workstation.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	frameBase64 : string
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes BroadcastScreen on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.BroadcastScreen (GET). • Appears in the <i>SEND MESSAGE TO STUDENT</i> module of the use case diagram.

T-177 STOP BROADCAST

Use Case Name	STOP BROADCAST
Purpose	Stop the teacher screen broadcast and return the workstations to the student view.
Primary Actor	Teacher
Secondary Actor	The CAMS client on the target workstation
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Teacher triggers the command from the monitoring console. 2. The browser invokes StopBroadcast on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.StopBroadcast (GET). • Appears in the <i>SEND MESSAGE TO STUDENT</i> module of the use case diagram.

MANAGE MONITORING ALERT TeacherController

T-178 VIEW ALERTS

Use Case Name	VIEW ALERTS
Purpose	List the monitoring alerts raised for the classes the teacher is responsible for.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	<code>filter : AlertListFilter</code>
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Alerts</code> (GET). • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram.

T-179 **ALERT HISTORY**

Use Case Name	ALERT HISTORY
Purpose	Show alerts that have already been acted on, with who acted and when.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by TeacherController.AlertHistory (GET). • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram. • Pulls in EXPORT ALERTS CSV (<<extend>>).

T-180 **OPEN ALERT COUNT**

Use Case Name	OPEN ALERT COUNT
Purpose	Report how many alerts are still open, for the badge on the navigation bar.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.OpenAlertCount (GET)</code>. • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram.

T-181 **ACKNOWLEDGE ALERT**

Use Case Name	ACKNOWLEDGE ALERT
Purpose	Mark one alert as seen and being handled.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int acknowledged : bool filter : AlertListFilter
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.AcknowledgeAlert</code> (POST). • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram.

T-182 **BULK ACKNOWLEDGE ALERTS**

Use Case Name	BULK ACKNOWLEDGE ALERTS
Purpose	Acknowledge several alerts in one action.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	alertIds : List<int>? filter : AlertListFilter
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by TeacherController.BulkAcknowledgeAlerts (POST). • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram.

T-183 **BULK DISMISS ALERTS**

Use Case Name	BULK DISMISS ALERTS
Purpose	Dismiss several alerts in one action.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	alertIds : List<int>? reason : string? filter : AlertListFilter
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>TeacherController.BulkDismissAlerts</code> (POST). • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram.

T-184 **BULK REOPEN ALERTS**

Use Case Name	BULK REOPEN ALERTS
Purpose	Reopen several alerts that were closed too early.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	alertIds : List<int>? filter : AlertListFilter
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The Teacher fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by TeacherController.BulkReopenAlerts (POST). • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram.

T-185 **EXPORT ALERTS CSV**

Use Case Name	EXPORT ALERTS CSV
Purpose	Produce the alerts view as a CSV file the user can download.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside ALERT HISTORY
Input Parameters	<code>filter : AlertListFilter</code>
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. ALERT HISTORY has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.ExportAlertsCsv (GET)</code>. • Appears in the <i>MANAGE MONITORING ALERT</i> module of the use case diagram. • Drawn as <<extend>> to ALERT HISTORY.

VIEW TEACHER RECORDS TeacherController

T-186 VIEW DASHBOARD

Use Case Name	VIEW DASHBOARD
Purpose	Show the teacher landing page with the state of the laboratory at a glance.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Dashboard (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram.

T-187 **VIEW RECORDS**

Use Case Name	VIEW RECORDS
Purpose	Show captured application and website activity for the teacher's students.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.Records (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Pulls in EXPORT RECORDS CSV (<<extend>>).

T-188 CLASS ANALYTICS

Use Case Name	CLASS ANALYTICS
Purpose	Show usage patterns for a class rather than for one student.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	<code>id : int</code> <code>from : DateTime? (optional)</code> <code>to : DateTime? (optional)</code> <code>station : string? (optional)</code>
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.ClassAnalytics (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram.

T-189 **LAB UTILIZATION**

Use Case Name	LAB UTILIZATION
Purpose	Show how heavily the laboratory workstations are being used over time.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) station : string? (optional) classId : int? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by TeacherController.LabUtilization (GET). • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram.

T-190 UNIFIED TIMELINE

Use Case Name	UNIFIED TIMELINE
Purpose	Show one student's activity as a single timeline across applications, websites and idle periods.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) studentId : int? (optional) classId : int? (optional) station : string? (optional) source : string? (optional) eventType : string? (optional) page : int (optional) pageSize : int (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.UnifiedTimeline (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram.

T-191 **ACTIVITY TIMELINE**

Use Case Name	ACTIVITY TIMELINE
Purpose	Serve the timeline data the unified timeline view is drawn from.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int from : DateTime? (optional) to : DateTime? (optional) page : int (optional) pageSize : int (optional) eventType : string? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by TeacherController.ActivityTimeline (GET). • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram.

T-192 **BROWSER MONITORING HISTORY**

Use Case Name	BROWSER MONITORING HISTORY
Purpose	Show the record of browser activity captured from the workstations.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) browser : string? (optional) mode : string? (optional) page : int (optional) pageSize : int (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.BrowserMonitoringHistory (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Pulls in EXPORT BROWSER MONITORING CSV (<<extend>>).

T-193 REMOTE HISTORY

Use Case Name	REMOTE HISTORY
Purpose	Show which remote commands were issued, by whom, and against which workstation.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) command : string? (optional) studentId : string? (optional) page : int (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.RemoteHistory (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Pulls in EXPORT REMOTE HISTORY CSV (<<extend>>).

T-194 STUDENT DETAILS

Use Case Name	STUDENT DETAILS
Purpose	Show one student in full: account, class, sessions and captured activity.
Primary Actor	Teacher
Secondary Actor	None
Input Parameters	id : int from : DateTime? (optional) to : DateTime? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.StudentDetails (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Pulls in EXPORT STUDENT ANALYTICS CSV (<<extend>>).

T-195 **EXPORT RECORDS CSV**

Use Case Name	EXPORT RECORDS CSV
Purpose	Produce the records view as a CSV file the user can download.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside VIEW RECORDS
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. VIEW RECORDS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.ExportRecordsCsv (GET)</code>. • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Drawn as <<extend>> to VIEW RECORDS.

T-196 **EXPORT REMOTE HISTORY CSV**

Use Case Name	EXPORT REMOTE HISTORY CSV
Purpose	Produce the remote history view as a CSV file the user can download.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside REMOTE HISTORY
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) command : string? (optional) studentId : string? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists. REMOTE HISTORY has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.ExportRemoteHistoryCsv</code> (GET). • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Drawn as <<extend>> to REMOTE HISTORY.

T-197 **EXPORT BROWSER MONITORING CSV**

Use Case Name	EXPORT BROWSER MONITORING CSV
Purpose	Produce the browser monitoring view as a CSV file the user can download.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside BROWSER MONITORING HISTORY
Input Parameters	from : DateTime? (optional) to : DateTime? (optional) browser : string? (optional) mode : string? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. BROWSER MONITORING HISTORY has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>TeacherController.ExportBrowserMonitoringCsv</code> (GET). • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Drawn as <<extend>> to BROWSER MONITORING HISTORY.

T-198 **EXPORT STUDENT ANALYTICS CSV**

Use Case Name	EXPORT STUDENT ANALYTICS CSV
Purpose	Produce the student analytics view as a CSV file the user can download.
Primary Actor	Teacher
Secondary Actor	None; this behaviour runs inside STUDENT DETAILS
Input Parameters	id : int from : DateTime? (optional) to : DateTime? (optional)
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a teacher. The record named by the identifier exists. STUDENT DETAILS has reached the point where this is optionally performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Teacher opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by <code>TeacherController.ExportStudentAnalyticsCsv</code> (GET). • Appears in the <i>VIEW TEACHER RECORDS</i> module of the use case diagram. • Drawn as <<extend>> to STUDENT DETAILS.

Student

PROCESS LOG IN AccountController

S-199 LOGIN USER

Use Case Name	LOGIN USER
Purpose	Let a person sign in to the CAMS web portal with a username and password, and place them in the part of the system their role allows.
Primary Actor	Student
Secondary Actor	None
Input Parameters	username : string password : string
Output Parameters	An authentication cookie carrying the account role as a claim, and a redirect to the landing page for that role.
Pre-condition	None. This is the sign-in endpoint and is reachable without an account session; the CAMS server must be running and reachable.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Student opens the CAMS sign-in page. 2. The Student enters a username and a password and submits the form. 3. The server validates the antiforgery token that accompanied the form. 4. The server checks the credentials against each account table in turn and finds the matching account. 5. The server confirms the account is active and not locked out. 6. The server issues an authentication cookie carrying the role as a claim. 7. The Student is redirected to the landing page for that role.
Exception Scenario	• The username matches no account — the page reports that the sign-in failed, without saying which half was wrong. • The password does not match the stored hash — the failed-attempt counter is raised and the same message is shown. • The account is locked out — the sign-in is refused until the lockout expires, even with the right password. • The account is deactivated — the sign-in is refused and the person is told to contact an administrator.
Additional Remarks	• Implemented by <code>AccountController.Login</code> (POST). • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram.

	• Pulls in LOGIN ASYNC (<<include>>).
--	--

S-200 **LOGOUT USER**

Use Case Name	LOGOUT USER
Purpose	End the signed-in session and clear the authentication cookie, so the next visitor to the browser starts as an anonymous user.
Primary Actor	Student
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The authentication cookie is cleared and the browser is returned to the sign-in page.
Pre-condition	The caller is signed in.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Student chooses to sign out. 2. The server closes any lab session the account still has open. 3. The server clears the authentication cookie. 4. The browser is returned to the sign-in page as an anonymous visitor.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>AccountController.Logout</code> (POST). • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Pulls in LOGOUT ASYNC (<<include>>).

S-201 **LOGIN ASYNC**

Use Case Name	LOGIN ASYNC
Purpose	Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.
Primary Actor	Student
Secondary Actor	None; this behaviour runs inside LOGIN USER
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A <code>LoginResult</code> naming the role, the account id and the display name, or a result saying the credentials were rejected.
Pre-condition	The sign-in use case has supplied a username and a password. LOGIN USER has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The sign-in use case passes the username, the password, the workstation name and the caller address. 2. The service looks the username up in the administrator, teacher and student tables. 3. The stored hash is verified against the supplied password. 4. The active flag and the lockout expiry are checked. 5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>LoginAsync</code>. • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Drawn as <<include>> from LOGIN USER.

S-202 **LOGOUT ASYNC**

Use Case Name	LOGOUT ASYNC
Purpose	Close any lab session the student still has open and record the sign-out.
Primary Actor	Student
Secondary Actor	None; this behaviour runs inside LOGOUT USER
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. LOGOUT USER has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs LogoutAsync and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by LogoutAsync. • Appears in the <i>PROCESS LOG IN</i> module of the use case diagram. • Drawn as <<include>> from LOGOUT USER.

LOG IN AT WORKSTATION ClientAuthController

S-203 CLIENT LOGIN

Use Case Name	CLIENT LOGIN
Purpose	Let a student sign in from the CAMS client installed on a laboratory workstation, binding the sign-in to the machine the student is sitting at.
Primary Actor	Student
Secondary Actor	None
Input Parameters	request : StudentClientLoginRequest
Output Parameters	A StudentClientLoginResponse carrying the student identity, the workstation registration and the session state.
Pre-condition	None. The client reaches this before any session exists. The workstation must have found the server and must trust the CAMS root certificate.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The CAMS client finds the server on the laboratory network. 2. The student enters a username and a password in the client. 3. The client posts the credentials together with the name of the workstation. 4. The server verifies the credentials and confirms the account is a student that is active and not locked out. 5. The server registers the workstation against the student, reassigning it if it was held by someone else who is no longer using it. 6. The client receives the session details and the monitored session begins.
Exception Scenario	• No server was found on the network — the client reports that it cannot reach CAMS and offers the endpoint to be entered by hand. • The workstation is already held by another active session — the sign-in is refused rather than displacing the student using it. • The account is not a student — the client refuses the sign-in; teacher and administrator accounts use the web portal.
Additional Remarks	• Implemented by <code>ClientAuthController.Login</code> (POST). • Appears in the <i>LOG IN AT WORKSTATION</i> module of the use case diagram. • Pulls in DEPLOYMENT PING (<<include>>), LOGIN ASYNC (<<include>>).

S-204 **CLIENT LOGOUT**

Use Case Name	CLIENT LOGOUT
Purpose	End the workstation session from the client, releasing the workstation so another student may sign in to it.
Primary Actor	Student
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	Confirmation that the session was closed and the workstation released.
Pre-condition	The caller is signed in as a student.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The student signs out from the CAMS client. 2. The server ends the lab session and records its end time. 3. The workstation is released so another student may sign in at it.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>ClientAuthController.Logout (POST)</code>. • Appears in the <i>LOG IN AT WORKSTATION</i> module of the use case diagram.

S-205 DEPLOYMENT PING

Use Case Name	DEPLOYMENT PING
Purpose	Answer the discovery request a client broadcasts while looking for the CAMS server on the laboratory network.
Primary Actor	Student
Secondary Actor	None; this behaviour runs inside CLIENT LOGIN
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The server endpoint a client should connect to.
Pre-condition	None. The endpoint answers an unauthenticated broadcast, so a client can find the server before it has credentials. CLIENT LOGIN has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. A CAMS client broadcasts on the laboratory network looking for a server. 2. The server answers with the endpoint the client should use. 3. The client stores the endpoint and proceeds to sign in.
Exception Scenario	• The broadcast does not reach the server — no answer is returned and the client falls back to a configured endpoint.
Additional Remarks	• Implemented by <code>DeploymentPingController.Get (GET)</code>. • Appears in the <i>LOG IN AT WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from CLIENT LOGIN.

S-206 **LOGIN ASYNC**

Use Case Name	LOGIN ASYNC
Purpose	Check the supplied password against the stored hash, confirm the account is active and not locked out, and decide which of the three account tables the person belongs to.
Primary Actor	Student
Secondary Actor	None; this behaviour runs inside CLIENT LOGIN
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A <code>LoginResult</code> naming the role, the account id and the display name, or a result saying the credentials were rejected.
Pre-condition	The sign-in use case has supplied a username and a password. CLIENT LOGIN has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The sign-in use case passes the username, the password, the workstation name and the caller address. 2. The service looks the username up in the administrator, teacher and student tables. 3. The stored hash is verified against the supplied password. 4. The active flag and the lockout expiry are checked. 5. On success the failed-attempt counter is cleared and the role and account id are returned; on failure the counter is raised.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>LoginAsync</code>. • Appears in the <i>LOG IN AT WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from CLIENT LOGIN.

WORK AT MONITORED WORKSTATION RemoteMonitoringHub

S-207 FETCH RESTRICTIONS

Use Case Name	FETCH RESTRICTIONS
Purpose	Give the client the restriction rules that apply to the student signed in at that workstation.
Primary Actor	Student
Secondary Actor	The CAMS client on the target workstation
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Student triggers the command from the monitoring console. 2. The browser invokes <code>FetchRestrictions</code> on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by <code>RemoteMonitoringHub.FetchRestrictions (GET)</code>. • Appears in the <i>WORK AT MONITORED WORKSTATION</i> module of the use case diagram. • Pulls in REPORT ACTIVE APP (<<include>>), REPORT WEBSITE ACTIVITY (<<include>>), REPORT IDLE STATUS (<<include>>), REPORT BROWSER MONITORING STATUS (<<include>>), REPORT TELEMETRY BATCH (<<include>>), REPORT INFRACTION (<<extend>>).

S-208 **REPORT ACTIVE APP**

Use Case Name	REPORT ACTIVE APP
Purpose	Report which application is in the foreground on the workstation.
Primary Actor	Student
Secondary Actor	The CAMS client on the target workstation
Input Parameters	app : ActiveAppMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. FETCH RESTRICTIONS has reached the point where this is always performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Student triggers the command from the monitoring console. 2. The browser invokes ReportActiveApp on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ReportActiveApp (GET). • Appears in the <i>WORK AT MONITORED WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from FETCH RESTRICTIONS.

S-209 **REPORT WEBSITE ACTIVITY**

Use Case Name	REPORT WEBSITE ACTIVITY
Purpose	Report the website the student is viewing in the browser.
Primary Actor	Student
Secondary Actor	The CAMS client on the target workstation
Input Parameters	website : WebsiteActivityMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. FETCH RESTRICTIONS has reached the point where this is always performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Student triggers the command from the monitoring console. 2. The browser invokes ReportWebsiteActivity on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ReportWebsiteActivity (GET). • Appears in the <i>WORK AT MONITORED WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from FETCH RESTRICTIONS.

S-210 **REPORT IDLE STATUS**

Use Case Name	REPORT IDLE STATUS
Purpose	Report whether the workstation has gone idle.
Primary Actor	Student
Secondary Actor	The CAMS client on the target workstation
Input Parameters	status : IdleStatusMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. FETCH RESTRICTIONS has reached the point where this is always performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Student triggers the command from the monitoring console. 2. The browser invokes ReportIdleStatus on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ReportIdleStatus (GET). • Appears in the <i>WORK AT MONITORED WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from FETCH RESTRICTIONS.

S-211 REPORT BROWSER MONITORING STATUS

Use Case Name	REPORT BROWSER MONITORING STATUS
Purpose	Report whether browser monitoring is working on the workstation.
Primary Actor	Student
Secondary Actor	The CAMS client on the target workstation
Input Parameters	status : BrowserMonitoringStatusMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. FETCH RESTRICTIONS has reached the point where this is always performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Student triggers the command from the monitoring console. 2. The browser invokes ReportBrowserMonitoringStatus on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ReportBrowserMonitoringStatus (GET). • Appears in the <i>WORK AT MONITORED WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from FETCH RESTRICTIONS.

S-212 REPORT TELEMETRY BATCH

Use Case Name	REPORT TELEMETRY BATCH
Purpose	Send a batch of buffered telemetry, so a brief disconnection does not lose the record.
Primary Actor	Student
Secondary Actor	The CAMS client on the target workstation
Input Parameters	batch : TelemetryBatchMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. FETCH RESTRICTIONS has reached the point where this is always performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Student triggers the command from the monitoring console. 2. The browser invokes ReportTelemetryBatch on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ReportTelemetryBatch (GET). • Appears in the <i>WORK AT MONITORED WORKSTATION</i> module of the use case diagram. • Drawn as <<include>> from FETCH RESTRICTIONS.

S-213 **REPORT INFRACTION**

Use Case Name	REPORT INFRACTION
Purpose	Report that the student tried to open something a restriction rule blocks.
Primary Actor	Student
Secondary Actor	The CAMS client on the target workstation
Input Parameters	infraction : InfractionMessage
Output Parameters	A SignalR message delivered to the target workstation or group; no HTTP response.
Pre-condition	The caller is signed in. FETCH RESTRICTIONS has reached the point where this is optionally performed.
Post-condition	The workstation has acted on the command and the console shows its new state.
Successful Scenario	1. The Student triggers the command from the monitoring console. 2. The browser invokes ReportInfraction on the SignalR hub over the open connection. 3. The server checks the caller's role and resolves the target connection. 4. The server relays the instruction to the workstation client. 5. The client carries it out and the console reflects the new state.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • The workstation is not connected — the command cannot be delivered and the console reports the workstation as offline.
Additional Remarks	• Implemented by RemoteMonitoringHub.ReportInfraction (GET). • Appears in the <i>WORK AT MONITORED WORKSTATION</i> module of the use case diagram. • Drawn as <<extend>> to FETCH RESTRICTIONS.

VIEW STUDENT SESSION `StudentController`S-214 **VIEW STUDENT HOME**

Use Case Name	VIEW STUDENT HOME
Purpose	Show the student portal home with the current session state and timer.
Primary Actor	Student
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a student.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Student opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>StudentController.Index (GET)</code>. • Appears in the <i>VIEW STUDENT SESSION</i> module of the use case diagram.

MANAGE STUDENT ALERT StudentController

S-215 VIEW ALERTS

Use Case Name	VIEW ALERTS
Purpose	List the monitoring alerts raised for the classes the teacher is responsible for.
Primary Actor	Student
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a student.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Student opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>StudentController.Alerts</code> (GET). • Appears in the <i>MANAGE STUDENT ALERT</i> module of the use case diagram.

S-216 MARK READ

Use Case Name	MARK READ
Purpose	Let a student mark one of their own alerts as read.
Primary Actor	Student
Secondary Actor	None
Input Parameters	id : int
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a student. The record named by the identifier exists.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Student opens the page and CAMS confirms the role on the authentication cookie. 2. The Student fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through ApplicationDbContext. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written. • The identifier matches no record — the action reports that the record was not found and makes no change.
Additional Remarks	• Implemented by StudentController.MarkRead (POST). • Appears in the <i>MANAGE STUDENT ALERT</i> module of the use case diagram.

MANAGE STUDENT ACCOUNT StudentController

S-217 VIEW SETTINGS

Use Case Name	VIEW SETTINGS
Purpose	Show the settings page for the signed-in user.
Primary Actor	Student
Secondary Actor	None
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The rendered page, or a JSON payload where the caller is the page's own script.
Pre-condition	The caller is signed in as a student.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The Student opens the page and CAMS confirms the role on the authentication cookie. 2. The server reads the records the caller is entitled to see. 3. The page renders with those records.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page.
Additional Remarks	• Implemented by <code>StudentController.Settings (GET)</code>. • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram.

S-218 **RESET PASSWORD**

Use Case Name	RESET PASSWORD
Purpose	Let a student replace their own password from the student portal.
Primary Actor	Student
Secondary Actor	None
Input Parameters	input : PasswordChangeInput
Output Parameters	A redirect back to the listing page, carrying a success or failure message for display.
Pre-condition	The caller is signed in as a student.
Post-condition	The change is committed to the database and visible to the next read. The caller sees the outcome reported on the page they return to.
Successful Scenario	1. The Student opens the page and CAMS confirms the role on the authentication cookie. 2. The Student fills the form and submits it. 3. The server validates the antiforgery token that accompanied the form. 4. The server validates the submitted values against the model rules. 5. The change is written to the database through <code>ApplicationDbContext</code>. 6. The server redirects back to the listing, where the result is shown.
Exception Scenario	• Not signed in or wrong role — the request is refused and the caller is sent to the access denied page. • Missing or stale antiforgery token — the submission is rejected and must be retried from a freshly loaded form. • Validation fails — the form is redisplayed with the offending fields marked and nothing is written.
Additional Remarks	• Implemented by <code>StudentController.ResetPassword</code> (POST). • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram. • Pulls in CHANGE STUDENT PASSWORD ASYNC (<<include>>).

S-219 **CHANGE STUDENT PASSWORD ASYNC**

Use Case Name	CHANGE STUDENT PASSWORD ASYNC
Purpose	Verify the current password and replace the stored hash for a student account.
Primary Actor	Student
Secondary Actor	None; this behaviour runs inside RESET PASSWORD
Input Parameters	None beyond the signed-in identity carried on the authentication cookie.
Output Parameters	The behaviour completes and its effect is visible to the use case that includes it.
Pre-condition	The caller is signed in. RESET PASSWORD has reached the point where this is always performed.
Post-condition	The caller has the requested information. Nothing in the database has changed.
Successful Scenario	1. The including use case reaches the point where this behaviour is required. 2. The server runs <code>ChangeStudentPasswordAsync</code> and applies its result. 3. Control returns to the including use case, which continues.
Exception Scenario	• The behaviour fails and the including use case reports the failure rather than continuing as if it had succeeded.
Additional Remarks	• Implemented by <code>ChangeStudentPasswordAsync</code>. • Appears in the <i>MANAGE STUDENT ACCOUNT</i> module of the use case diagram. • Drawn as <<include>> from RESET PASSWORD.